

Splicing a base class subobject

Document #: P3293R2 [[Latest](#)] [[Status](#)]
Date: 2025-05-01
Project: Programming Language C++
Audience: CWG/LWG
Reply-to: Peter Dimov
<pdimov@gmail.com>
Dan Katz
<dkatz85@bloomberg.net>
Barry Revzin
<barry.revzin@gmail.com>
Daveed Vandevoorde
<daveed@edg.com>

Contents

- [1 Revision History](#)
- [2 Introduction](#)
- [3 Kinds of Subobjects](#)
 - [3.1 Base Class Subobjects](#)
 - [3.2 Array Elements](#)
- [4 Proposal](#)
 - [4.1 Wording](#)
- [5 References](#)

§ 1 Revision History

Since [\[P3293R1\]](#), updating wording and design to account for [\[P3547R1\]](#) ([Modeling Access Control With Reflection](#)). Adding corresponding `has_inaccessible_subobjects`.

Since [\[P3293R0\]](#), noted that `&[:base:]` cannot work for virtual base classes. Talking about arrays. Added wording.

§ 2 Introduction

There are many contexts in which it is useful to perform the same operation on each subobject of an object in sequence. These include serialization or [formatting](#) or [hashing](#).

[\[P2996R6\]](#) seems like it gives us an ideal solution to this problem, in the form of being able to iterate over all the subobjects of an object and splicing accesses to them. However, it is not quite complete:

```

template <class T, class F>
void for_each_subobject(T const& obj, F f) {
    template for (constexpr auto sub : subobjects_of(^T)) {
        f(obj.[:sub:]); // this is valid syntax for non-static data
                        members
                        // but is invalid for base classes subobjects
    }
}

```

Instead we have to handle bases separately from the non-static data members:

```

template <class T, class F>
void for_each_subobject(T const& obj, F f) {
    template for (constexpr auto base : bases_of(^T)) {
        f(static_cast<type_of(base) const&>(obj));
    }

    template for (constexpr auto sub : nonstatic_data_members_of(^T)) {
        f(obj.[:sub:]);
    }
}

```

Except this is now a normal `static_cast` and so requires access checking, thus prohibiting accessing private base classes.

We could avoid access checking by using a C-style cast:

```

template <class T, class F>
void for_each_subobject(T const& obj, F f) {
    template for (constexpr auto base : bases_of(^T)) {
        f((typename [: type_of(base) :]&)obj);
    }

    template for (constexpr auto sub : nonstatic_data_members_of(^T)) {
        f(obj.[:sub:]);
    }
}

```

But this opens up other problems: I forgot to write `const` and so now I accidentally cast away `const`-ness unintentionally. Oops. Not to mention that this cast actually works regardless of whether base refers to a base class of `T`, so it's not exactly the best programming practice.

On top of that, both the `static_cast` and C-style cast approaches suffer from having to correctly spell the destination type - which requires manually propagating the `const`-ness and value category of the object.

The way to avoid all of these problems is to just defer to a function template:

```

template <std::meta::info M, class T>
constexpr auto subobject_cast(T&& arg) -> auto&& {
    constexpr auto stripped = remove_cvref(^T);
    if constexpr (is_base(M)) {
        static_assert(is_base_of(type_of(M), stripped));
        return (typename [: copy_cvref(^T, type_of(M)) :]&)arg;
    } else {
        static_assert(parent_of(M) == stripped);
        return ((T&&)arg).[:M:];
    }
}

template <class T, class F>
void for_each_subobject(T const& obj, F f) {
    template for (constexpr auto sub : subobjects_of(^T)) {
        f(subobject_cast<sub>(obj));
    }
}

```

But this feels a bit silly? Why should we have to write this?

§ 3 Kinds of Subobjects

There are three kinds of subobjects as specified in 6.7.2 [\[intro.object\]](#):

- 2 Objects can contain other objects, called *subobjects*. A subobject can be a *member subobject* ([class.mem]), a *base class subobject* ([class.derived]), or an array element.

Currently, the reflection proposal only supports splicing member subobjects. Let's go over the other two.

§ 3.1 Base Class Subobjects

Unlike member subobjects, there is *no* way today to access a base class subobject directly outside of one of the casts described above. Part of the reason for this is that while a data member is always just an *identifier*, a base class subobject can have an arbitrary complex name. Reflection allows us to sidestep the question of complex names since we just have a reflection to the appropriate base class subobject already, so it becomes a matter of asking what `obj.[:base:]` means.

But what else could it mean? We argue that the obvious, useful, and only possible meaning of this syntax would be to access the appropriate base class subobject (in the same way that `obj.[:nsdm:]` — where `nsdm` represents a non-static data member — is access to that data member).

Additionally, `&[:base:]` where `base` represents a base class `B` of type `T` could yield a `B T::*` with appropriate offset. Likewise, there is no way to directly get such a member pointer today. But that would be a useful bit of functionality to add. That is, unless `base` represents a reflection of a virtual base class subobject, which wouldn't be representable as a pointer to member.

The only reason [\[P2996R6\]](#) doesn't support splicing base class subobjects is the lack of equivalent language support today. This means that adding this support in reflection would mean that splicing can achieve something the language cannot do natively. But we don't really see that as a problem. Reflection is already allowing all sorts of things that the language cannot do natively. What's one more?

§ 3.2 Array Elements

The more complicated question is array elements. `subobjects_of(^int[4])` needs to return four subobjects, which would effectively each represent “index `i` into `int[4]`” for each `i` in `[0, 1, 2, 3]`. We would want both the index and the type here. That, in of itself, isn't any different from dealing with the non-static data members of a class. These subobjects likewise have offsets, types, etc, in the same way.

The challenge is more in terms of access. With base class subobjects, we talked about how there is no non-splice equivalent to `obj.[[:base:]]`. But today there isn't any even any such valid syntax for arrays today at all! Indeed, the standard refers to the expression `E1.E2` as *class member access* (7.6.1.5 [\[expr.ref\]](#)), and arrays are not actually classes. That adds complexity.

Moreover, arrays have the additional problem that the number of array elements can explode quite quickly. `int[1'000]` already has 1000 subobjects. That's expensive. Especially since array elements aren't quite the same as the other kinds of subobjects — we know they are all the same type. All the use-cases that we have, unlike for base class subobjects, would want to treat arrays separately anyway.

That is, being able to splice base class subobjects is small language extension with good bang for the buck. Being able to splice array subobjects requires being able to even represent array subobjects as reflections (which currently does not exist) as well as being able to extend class member access to support arrays. Not to mention pointers to members? It's a much larger change with a much smaller benefit.

§ 4 Proposal

We propose two language changes:

1. to define `obj.[[:base:]]` (where `base` is a reflection of a base class of the type of `obj`) as being an access to that base class subobject, in the same way that `obj.[[:nsdm:]]` (where `nsdm` is a reflection of a non-static data member) is an access to that data member.
2. to define `&[[:base:]]` where `base` is a reflection of a base class `B` of type `T` should yield a `B T::*` with appropriate offset. Unless `base` is a reflection of a virtual base class, which wouldn't really be representable as a pointer to member.

We argue that these are the obvious, useful, and only possible meanings of these syntaxes, so we should simply support them in the language.

The only reason this isn't initially part of [\[P2996R6\]](#) is that while there *is* a way to access a data member of an object directly (just `obj.mem`), there is *no* way to access a base class subobject directly outside of one of the casts described above.

We then additionally propose to back `subobjects_of()` that [P2996R6] removed. This was removed because iterating over *all* the subobjects uniformly wasn't really possible until these language additions.

§ 4.1 Wording

The wording here is a diff on top of P2996.

Adjust the *splice-expression* restriction added by P2996:

- * If E2 is a *splice-expression*, then let T1 be the type of E1. E2 shall designate either a member of the type of E1 T1 or a direct base class relationship whose derived class is T1.

Handle base class splices in 7.6.1.5 [expr.ref]/7-8:

- 7 If E2 designates an entity that is declared to have type “reference to T”, then E1.E2 is an lvalue of type T. In that case, if E2 designates a static data member, E1.E2 designates the object or function to which the reference is bound, otherwise E1.E2 designates the object or function to which the corresponding reference member of E1 is bound. Otherwise, one of the following rules applies.
 - (7.1) — If E2 designates a static data member and the type of E2 is T, then E1.E2 is an lvalue; [...]
 - (7.2) — Otherwise, if E2 designates a non-static data member and the type of E1 is “*cq1 vq1 X*”, and the type of E2 is “*cq2 vq2 T*”, the expression designates the corresponding member subobject of the object designated by the first expression. [...]
 - (7.3) — Otherwise, if E2 is an overload set, [...]
 - (7.4) — Otherwise, if E2 designates a nested type, the expression E1.E2 is ill-formed.
 - (7.5) — Otherwise, if E2 designates a member enumerator [...]
 - (7.6) — Otherwise, if E2 designates a direct, non-virtual base class relationship with base class *B*, the expression designates the base class subobject of type *B* corresponding to the the object designated by the first expression. If E1 is an lvalue, then E1.E2 is an lvalue; otherwise E1.E2 is an xvalue. The type of E1.E2 is “*cv B*”. [*Note 1*: This can only occur in an expression of the form `e1.[:e2:]` where `e2` is a reflection designating a base class subobject. — *end note*]

[*Example 1*:

```
struct B {
    int b;
};
struct D : B {
    int d;
};
```

```
constexpr int f() {
    D d = {1, 2};
    B& b = d.[: std::meta::bases_of(^^D,
        std::meta::access_context::current())[0] :];
    b.b += 10;
    d.[: ^^D::d :] += 1;
    return d.b * d.d;
}
static_assert(f() == 33);
```

— end example]

(7.7) — Otherwise, the program is ill-formed.

Handle base class pointers to members in 7.6.2.2 [\[expr.unary.op\]](#):

3 The operand of the unary & operator shall be an lvalue of some type T.

(3.1) — If the operand is a *qualified-id* or *splice-expression* designating a non-static or variant member of some class C, other than an explicit object member function, the result has type “pointer to member of class C of type T” and designates C::m.

(3.1+) — Otherwise, if the operand is a *splice-expression* designating a direct base class relationship of some class C with direct base class T, other than a virtual base class relationship, the result has type pointer to member of class C of type T and designates that base class subobject.

[Example 2:

```
struct B {
    int b;
};

struct D : B {
    int d;
};

constexpr D d = {1, 2};

constexpr int B::*pb = &[: std::meta::bases_of(^^D,
    std::meta::access_context::current())[0] :];
static_assert(d.*pb == 1);
static_assert(&(d.*pb) == &static_cast<B&>(d));
```

— end example]

(3.2) — Otherwise, the result has type “pointer to T” and points to the designated object (6.7.1 [\[intro.memory\]](#)) or function (6.8.4 [\[basic.compound\]](#)). If the operand designates an explicit object member function (9.3.4.6 [\[dcl.fct\]](#)), the operand shall be a *qualified-id* or a *splice-expression*.

- 4 A pointer to member is only formed when an explicit `&` is used and its operand is a *qualified-id* or *splice-expression* not enclosed in parentheses.

Add to meta.synop:

```
namespace std::meta {
    // ...

    // [meta.reflection.access.queries], member accessibility queries
    consteval bool is_accessible(info r, access_context ctx);
    consteval bool has_inaccessible_nonstatic_data_members(
        info r,
        access_context ctx);
    consteval bool has_inaccessible_bases(info r, access_context ctx);
+   consteval bool has_inaccessible_subobjects(info r, access_context
        ctx);

    // [meta.reflection.member.queries], reflection member queries
    consteval vector<info> members_of(info r, access_context ctx);
    consteval vector<info> bases_of(info type, access_context ctx);
        consteval vector<info> static_data_members_of(info type,
            access_context ctx);
        consteval vector<info> nonstatic_data_members_of(info type,
            access_context ctx);
+   consteval vector<info> subobjects_of(info type, access_context ctx);
    consteval vector<info> enumerators_of(info type_enum);

    // ...
}
```

Add to [meta.reflection.access.queries] in the appropriate spot:

- ```
consteval bool has_inaccessible_bases(info r, access_context ctx);
```
- 7 *Constant When:* `bases_of(r, ctx)` is a constant subexpression.
- 8 *Returns:* `true` if `is_accessible(R, ctx)` is `false` for any `R` in `bases_of(r, access_context::unchecked())`. Otherwise, `false`.
- ```
consteval bool has_inaccessible_subobjects(info r, access_context ctx);
```
- 9 *Effects:* Equivalent to: `return has_inaccessible_bases(r, ctx) || has_inaccessible_nonstatic_data_members(r, ctx);`

Add to [meta.reflection.member.queries] in the appropriate spot:

```
consteval vector<info> nonstatic_data_members_of(info type,
    access_context ctx);
```

9 *Constant When:* `dealias(type)` represents a complete class type.

10 *Returns:* A vector containing each element `e` of `members_of(type, ctx)` such that `is_nonstatic_data_member(e)` is `true`, preserving their order.

```
constexpr vector<info> subobjects_of(info type, access_context ctx);
```

10+1 *Constant When:* `dealias(type)` represents a complete class type.

10+2 *Returns:* A vector containing each element of `bases_of(type, ctx)` followed by each element of `nonstatic_data_members_of(type, ctx)`, preserving their order.

```
constexpr vector<info> enumerators_of(info type_enum);
```

11 *Constant When:* `dealias(type_enum)` represents an enumeration type and `has_complete_definition(dealias(type_enum))` is `true`.

12 *Returns:* A vector containing the reflections of each enumerator of the enumeration represented by `dealias(type_enum)`, in the order in which they are declared.

§ 5 References

[P2996R6] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. 2024-10-10. Reflection for C++26.

<https://wg21.link/p2996r6>

[P3293R0] Barry Revzin, Peter Dimov, Dan Katz, Daveed Vandevoorde. 2024-05-20. Splicing a base class subobject.

<https://wg21.link/p3293r0>

[P3293R1] Barry Revzin, Peter Dimov, Dan Katz, Daveed Vandevoorde. 2024-10-13. Splicing a base class subobject.

<https://wg21.link/p3293r1>

[P3547R1] Dan Katz, Ville Voutilainen. 2025-02-09. Modeling Access Control With Reflection.

<https://wg21.link/p3547r1>